
Cadence — Technical Implementation Document

Real-time USDC payment streaming on Arc L1

Prepared as an engineering reference and live-demo defence guide. It covers the smart contract and backend in depth, and the front-end only where it reads from the chain or the database. After reading this you should be able to explain any shipped behaviour, justify every design decision, and answer hard questions about correctness, security, and why Arc specifically makes the product possible.

1. What Cadence is, in one paragraph

Cadence lets a payer fund a stream of USDC once and have it flow to a payee continuously, by the second. The payer deposits the full amount up front into a smart contract; the contract releases it second by second at an agreed rate; the payee withdraws whatever has accrued whenever they want. Money that has been earned is money the payee already owns on-chain, not a promise waiting on a pay run. The payer can top up a live stream or cancel it at any time, and every unstreamed cent returns to their wallet on cancel. A payee can also request a stream, which the payer accepts as-is or counters, giving a lightweight on-chain negotiation. Everything that moves money lives in the contract; an off-chain database holds only conveniences like usernames and saved payees.

Live app: <https://cadenceonarc.tech>

Chain: Arc L1 testnet, chain id [5042002](#)

Deployed contract: [PayrollManager](#) at [0x1EAb59E203f080fa3c4b87C50d59eA9efD0f85e5](#)

USDC (native + ERC-20 interface): [0x3600000000000000000000000000000000000000](#)

2. Architecture at a glance

```
Browser (Next.js 15 App Router, React 19, TypeScript)
|
|-- Reads from chain -----> wagmi/viem --> same-origin /api/rpc proxy --> Arc RPC upstreams
|                                     (Multicall3 batching, 3-5s polling)
|
|-- Reads/writes off-chain -> /api/* routes (serverless, Node runtime)
|                                     --> Privy token verify --> Neon Postgres via Drizzle
|
|-- Writes to chain -----> wallet (Privy embedded or injected) --> Arc
|                                     (static gas limits + pinned EIP-1559 fees)

Smart contract (Solidity, Foundry): PayrollManager holds every stream's
escrow and does all per-second accounting on-chain.
```

Two hard rules shape the whole system:

1. **Nothing that determines who owns money lives off-chain.** Balances, accrual, escrow, refunds, and the request lifecycle are all contract state. The database can be wiped without anyone losing a cent.

2. **The client never trusts itself for money.** Every figure the UI shows (accrued, runway, refund on cancel) is *derived* from the on-chain struct, and every mutation is a contract call the wallet signs. The animated ticker is cosmetic interpolation between real reads.

3. Why Arc, specifically

Cadence is not a generic streaming app that happens to deploy on Arc. Three Arc protocol properties are load-bearing; remove any one and the product stops making sense. (All figures below are from the Arc docs, cited so they can be defended.)

3.1 USDC is the native gas token

On Arc, the asset you deposit, the asset that streams, and the gas that moves it are the **same** asset. Native gas accounting uses 18 decimals; the ERC-20 interface used by applications uses 6 decimals; they share one underlying balance (Arc docs: *Stablecoin native model*; *Contract addresses* → *USDC*).

Why this matters for Cadence: on any other EVM chain, a payee earns a stablecoin but needs a *separate, volatile* gas token (ETH, or the chain's native coin) just to withdraw it. Someone paid only in USDC would hold a balance they cannot touch without first going to acquire ETH. Arc collapses money and gas into one asset, so **earnings are always spendable**. A payee withdraws USDC by spending a sliver of that same USDC.

3.2 Sub-second deterministic finality

Arc's Malachite BFT consensus finalizes every block in under one second with **no reorganizations**; a committed transaction is final on commit (Arc docs: *Deterministic finality*). Compare: Ethereum L1 finality is 12-15 minutes; a typical optimistic L2 gates withdrawal finality behind a challenge window of roughly seven days.

Why this matters: "withdraw what you earned, right now" is incoherent if settlement is probabilistic or delayed. Cash-out-anytime is only *real* when finality is measured in milliseconds. On the UX side, the app awaits the receipt (`waitForTransactionReceipt`) and then fires notifications and refetches — because finality is instant, that flow feels immediate rather than "pending".

3.3 Stable, dollar-denominated fees

Fees are priced in USDC. Arc uses EIP-1559 with EWMA smoothing of block utilization, so short traffic spikes don't cause fee jumps; the base fee targets about a cent and returns to target quickly (Arc docs: *Gas and fees*; *Stable fee design*). Testnet enforces a **minimum base fee of 20 Gwei**.

Why this matters: streaming invites lots of tiny actions — withdraw a little, top up, cancel. On a volatile gas market a micro-withdrawal can cost more than it's worth. Arc's stable, dollar-priced fees keep those actions economical. (The 20 Gwei floor also directly shapes our fee strategy — see §7.2.)

3.4 Canonical USDC system events

On Arc every balance change emits a standard `Transfer` log (Arc's implementation of EIP-7708). That gives receipts and statements one universal source of truth for value movement, independent of our own contract's events.

Demo one-liner: *"Single-asset USDC for money and gas, sub-second final settlement, and stable sub-cent fees — those three together turn 'paid by the second, spendable by the second' from a demo into something*

that actually works. No other EVM chain gives you all three."

4. The smart contract: `PayrollManager`

Solidity `^0.8.20` , built and tested with Foundry (`optimizer = true` , `optimizer_runs = 200` , `evm_version = "prague"`). Single contract, no proxies, no admin, no owner. It holds every stream's escrow and does all accounting. Its only external dependency is the USDC ERC-20 interface and a minimal in-repo `ReentrancyGuard` .

4.1 Design principles

- **No privileged roles.** There is no owner, no pause, no upgrade path. The contract cannot be changed after deploy, and no one (including the deployer) can move a user's funds. This is a deliberate trust property: the only actors are the employer and employee of each stream.
- **Escrow up front.** Funds are always pulled into the contract *before* a stream is recorded, so a stream can never promise money the contract does not hold.
- **Derive, don't store, the moving figure.** Accrued value is computed from `(now - lastClaimTime) * ratePerSecond` , capped at the remaining deposit. Nothing needs to be written every second.
- **Checks-effects-interactions + reentrancy guard on every state-changing money path.**

4.2 The `Stream` data model

```
struct Stream {
    address employer;           // payer; the only one who can top up / cancel
    address employee;           // payee; the only one who can withdraw
    uint128 ratePerSecond;       // USDC/sec, 6 decimals (1e6 = $1/sec)
    uint64  startTime;           // when accrual begins (supports scheduling)
    uint64  lastClaimTime;        // accrual anchor; moves on withdraw/topUp/cancel
    uint128 deposit;              // REMAINING escrow: shrinks on withdraw, grows on topUp, 0 on cancel/drain
    uint128 totalDeposited;       // STATIC lifetime commitment: first deposit + all top-ups; only grows
    uint128 withdrawn;           // cumulative paid to employee (monotonic)
    bool    active;
    string  invoiceRef;          // arbitrary invoice/tax memo
}
```

The three money fields encode a full history without any event indexing:

- `deposit` is what's left in escrow *right now*.
- `totalDeposited` is the lifetime commitment (needed to distinguish "fully claimed" from "cancelled" — see §6.4).
- `withdrawn` is the running total actually paid out.

`ratePerSecond` , `deposit` , `totalDeposited` , `withdrawn` are `uint128` ; timestamps are `uint64` . This is both gas-efficient (struct packing) and safe: `uint128` covers far more USDC than will ever exist, and `uint64` covers timestamps for billions of years. All arithmetic runs under Solidity 0.8 checked math, so any overflow reverts rather than wrapping.

Streams are stored in `mapping(uint256 => Stream) streams` , with per-party index arrays `employerStreams[addr]` and `employeeStreams[addr]` and a monotonic `nextStreamId` . The index arrays are what the dashboards read to list "my streams" (see §6.1).

4.3 Accrual — the heart of the contract

```
function _accrued(Stream storage s) internal view returns (uint128) {
    if (!s.active) return 0;
    if (block.timestamp <= s.lastClaimTime) return 0; // scheduled/not-started guard
    uint64 elapsed = uint64(block.timestamp) - s.lastClaimTime;
    uint128 owed = uint128(elapsed) * s.ratePerSecond;
    return owed > s.deposit ? s.deposit : owed; // cap at remaining escrow
}
```

Two invariants live here:

1. **Never accrue past the escrow.** `owed` is capped at `deposit`, so the contract can never owe more than it holds. When elapsed time covers the whole balance, the next withdrawal takes *all* of it.
2. **Never underflow on a scheduled stream.** If `block.timestamp <= lastClaimTime` (a stream scheduled to start in the future), it returns 0 instead of subtracting a future timestamp.

Public views `accrued(streamId)` and `runway(streamId)` wrap this. `runway = (deposit - accrued) / ratePerSecond` = seconds of pay left at the current rate.

4.4 Creating a stream

```
function createStream(address employee, uint128 ratePerSecond, uint128 deposit,
    string calldata invoiceRef, uint64 startAt)
    external nonReentrant returns (uint256 streamId)
```

Flow: validate terms → `usdc.transferFrom(payer, contract, deposit)` → record the stream. Requires the payer to have approved USDC first (standard ERC-20 allowance).

`_validateTerms` enforces:

- `employee != address(0)`
- `ratePerSecond > 0`
- `deposit > 0`
- `deposit >= ratePerSecond` — the deposit must cover *at least one second* of pay. This blocks nonsensical streams that would be dead on arrival.

`startAt` enables **scheduling**: a zero or past value clamps to `now` (start immediately); a future value schedules the stream (funds escrowed now, nothing accrues until `startAt`). Scheduling is capped at `now + 365 days` so a stream can't be parked absurdly far out. On creation `lastClaimTime` is set equal to `startTime`, which is exactly why `_accrued` returns 0 until the start passes.

4.5 Batch creation — one signature, many streams

```
function createStreams(address[] employees, uint128[] ratesPerSecond, uint128[] deposits,
    string[] invoiceRefs, uint64 startAt)
    external nonReentrant returns (uint256[] streamIds)
```

This is a genuinely important design point and a good demo question to be ready for: **why is batching in the contract instead of a generic multicall?**

Because `createStream` binds both the employer *and* the funding source to `msg.sender`. If you routed a batch through a generic multicall/aggregator contract, that aggregator would become `msg.sender` — it would be

recorded as the employer of every stream and the funds would have to come from it. Batching therefore has to live *inside* `PayrollManager` so the payer stays the caller for every stream.

Properties:

- **All-or-nothing.** Every row is validated and the total is summed *before any funds move*. One bad row reverts the whole batch — the payer never ends up with a half-created set.
- **One transfer.** The entire batch's escrow is pulled in a single `transferFrom` of the summed total, so the payer approves once and signs the batch once (two signatures total regardless of recipient count).
- **Shared start.** All streams in a batch share one `startAt`, so you can schedule an entire payroll run to begin together.

4.6 Withdraw

```
function withdraw(uint256 streamId) external nonReentrant
```

Only the employee can call. Computes `owed = _accrued(s)`, requires `owed > 0`, then:

- `deposit -= owed`
- `withdrawn += owed`
- `lastClaimTime = block.timestamp` (re-anchor accrual clock)
- **deactivate only when** `deposit == 0` — a stream with runway left stays active regardless of rate.
- `usdc.transfer(employee, owed)`

No-dust guarantee (defensible correctness claim): because `_accrued` caps at the whole remaining deposit, once elapsed time covers the balance the withdrawal takes *everything*. No sub-second remainder can strand in the contract. This is tested explicitly (`test_NoDustStranded_FullDrainIsClaimable`, `test_ContractNeverStrandsFunds_Invariant`).

4.7 Top-up — the subtlest logic in the contract

```
function topUp(uint256 streamId, uint128 amount) external nonReentrant
```

Only the employer can call. It pulls `amount` in, then branches on the stream's phase. This is the part most worth understanding cold, because "top up" means different things depending on where the stream is:

Running stream (active and past its start): hold the finish date fixed, RAISE the rate.

The intuition: if you add money to a live payroll stream, you almost always mean "pay them more over the same period," not "pay them the same and drag the end date out." So:

1. Snapshot `owed = _accrued(s)` (already earned, not yet claimed) and `unstreamed = deposit - owed` (future money before this top-up).
2. Compute the remaining runway in seconds at the current rate: `unstreamed / ratePerSecond`.
3. Spread `(unstreamed + amount)` across those same remaining seconds → a strictly higher `newRate`.
4. Add the funds (`deposit += amount`, `totalDeposited += amount`) and set the new rate.
5. **Re-anchor the accrual clock** so the already-owed amount is unchanged under the new rate: `lastClaimTime = now - (owed / newRate)`. Because the rate only ever goes *up*, `owed / newRate <=`

`(now - oldLastClaimTime)` , so this can never underflow and never predates `startTime` .

Scheduled stream (active but not yet started): additive. The deposit grows; rate and start are untouched, so the extra funds simply extend the runway past the original end. Re-anchoring here would corrupt the schedule, so it's deliberately avoided before a stream begins.

Drained/inactive stream: reactivate and restart the clock from now. `deposit += amount` ; if the stream was inactive and now holds at least one second of pay, set `active = true` and `lastClaimTime = now` , so the idle gap while it sat empty is never retroactively owed.

The rate-raising branch has bounded sub-second dust, always in the employer's favour, and this is tested (`test_TopUp_RunningRaisesRatePreservesFinish` , `test_TopUp_RunningPreservesOwedForClaim` , `test_TopUp_RunningDustBoundedInEmployerFavour` , `test_TopUp_ScheduledIsAdditive` , `test_TopUp_FullyAccruedKeepsRateAddsRunway`).

4.8 Cancel

```
function cancelStream(uint256 streamId) external nonReentrant
```

Only the employer, only while active. It splits the escrow cleanly:

- `owed = _accrued(s)` goes to the **employee** (they keep everything earned up to this instant).
- `refund = deposit - owed` returns to the **employer**.
- Records the final payout (`withdrawn += owed`), advances `lastClaimTime` , sets `active = false` and `deposit = 0` , then transfers both legs.

The post-cancel state — `active == false` , `deposit == 0` , `withdrawn < totalDeposited` — is a distinct signature the front-end uses to render "cancelled" versus "fully claimed" (see §6.4).

4.9 The request / negotiation state machine

A payee can ask a payer to open a stream. This is a small on-chain state machine so neither side has to trust the other and no off-chain coordination is needed.

```
enum ReqStatus { Pending, Countered, Accepted, Rejected, Cancelled, Expired }
```

`StreamRequest` mirrors the stream terms (payee, payer, rate, deposit, startAt) plus a `counterDeadline` , a `status` , and the resulting `streamId` once accepted. Indexed by `payerRequests[addr]` (incoming) and `payeeRequests[addr]` (outgoing).

Transitions and who can trigger them:

Action	Caller	Funds movement	Result
<code>requestStream</code>	payee	none	<code>Pending</code>
<code>acceptRequest</code>	payer	payer <code>transferFrom</code> deposit	<code>Accepted</code> → opens stream
<code>counterRequest</code>	payer	payer escrows new deposit	<code>Countered</code> (6h deadline)
<code>acceptCounter</code>	payee	none (already escrowed)	<code>Accepted</code> → opens stream instantly

Action	Caller	Funds movement	Result
<code>rejectCounter</code>	payee	refund escrow to payer	<code>Rejected</code>
<code>reclaimExpiredCounter</code>	anyone	refund escrow to payer	<code>Expired</code>
<code>rejectRequest</code>	payer	none	<code>Rejected</code>
<code>cancelRequest</code>	payee	none	<code>Cancelled</code>

The clever bit is the **counter-offer escrow**. When a payer counters, they escrow the new deposit *immediately*. That means the payee can accept the counter with a single call and the stream opens with **no further payer signature** — the funds are already committed. To bound that commitment, a counter auto-expires after `COUNTER_WINDOW = 6 hours`.

Because a contract cannot fire its own refund when a deadline passes, `reclaimExpiredCounter` is **permissionless**: anyone may call it after the window lapses to return the escrowed funds to the payer. In practice the payer (or the payee, or any observer) triggers it; the refund always goes to the payer regardless of who calls.

`Accepted`, `Rejected`, `Cancelled`, `Expired` are terminal. `acceptRequest` and `acceptCounter` both funnel into the same `_openStream` helper that `createStream` uses, so an accepted request produces an ordinary stream indistinguishable from a directly-created one.

4.10 Events

Every state change emits an event: `StreamCreated`, `Withdrawn`, `StreamToppedUp`, `StreamCancelled`, and the request lifecycle events (`RequestCreated`, `RequestCountered`, `RequestAccepted`, `RequestRejected`, `RequestCancelled`, `RequestExpired`). Indexed by stream/request id and the relevant addresses. The app does **not** currently run an event indexer for its live views (it reads struct state directly and polls — see §6), but the events make the contract fully auditable on ArcScan and are the natural hook for a future indexer.

4.11 Security posture (be ready to defend this)

- **Reentrancy**: a minimal `ReentrancyGuard` (`_status` 1/2 flag) guards every money-moving function (`createStream`, `createStreams`, `withdraw`, `topUp`, `cancelStream`, `acceptRequest`, `counterRequest`, `acceptCounter`, `rejectCounter`, `reclaimExpiredCounter`). The read-only lifecycle transitions that move no funds (`rejectRequest`, `cancelRequest`) don't need it.
- **Checks-effects-interactions**: state is updated before external USDC transfers in every path (e.g. `withdraw` zeroes/decrements and re-anchors before `usdc.transfer`).
- **Access control by construction**: every mutating call requires `msg.sender` to be the correct party (`require(msg.sender == s.employer / s.employee / r.payer / r.payee)`). There are no roles to misconfigure.
- **No overflow**: Solidity 0.8 checked arithmetic throughout; `uint128` / `uint64` sizing leaves enormous headroom.
- **No stranded funds**: the accrual cap + full-drain withdrawal is proven by an invariant test.
- **No admin surface**: nothing to pause, upgrade, or drain. The deployer has no special powers.

- **Griefing bounded:** the only cross-party time dependency, the counter-offer, has a fixed 6-hour window and a permissionless reclaim, so escrow can't be locked indefinitely.

4.12 Test coverage

40 Foundry tests in `contracts/test/PayrollManager.t.sol`, covering: basic create/accrue/withdraw/cancel; runway math; multi-claim accumulation; `totalDeposited` static-across-withdraw and growing-on-topup; all three top-up branches and dust bounds; scheduling (no accrual before start, accrual after, withdraw reverts before start, cancel-before-start refunds all, past-start clamps, too-far-out reverts); batch create (opens all + single pull, atomic revert on a bad row, length-mismatch revert, shared scheduled start); the full request lifecycle (create, accept, counter+escrow, accept-counter-instant, reject-counter-refund, reclaim-expired-refund, reject, cancel, self-request revert, only-payer/only-payee guards, after-window revert); and the two safety invariants (no dust stranded, contract never strands funds).

Run with `forge test` (or `forge test --gas-report` to regenerate the numbers the front-end's static gas limits are based on — see §7.1).

4.13 Deployment

`contracts/script/Deploy.s.sol` reads `PRIVATE_KEY` from the environment and deploys `new PayrollManager(0x3600...0000)` — passing the Arc USDC system-contract address into the constructor, which stores it as the immutable `usdc`. Deployed to Arc testnet at `0x1EAb59E203f080fa3c4b87C50d59eA9efD0f85e5`.

Operational note worth knowing for the demo: the front-end reads the contract address from `NEXT_PUBLIC_PAYROLL_ADDRESS`, which is baked in at build time. If the contract is redeployed, the env var must change *and* the app must be rebuilt/redeployed to point at it. A redeploy is a fresh contract with empty state, so it does not carry existing streams over.

5. Off-chain backend: identity and conveniences only

The database exists purely to make the app pleasant — human-readable handles, an address book, saved drafts, notification email. **Nothing that moves money or is a balance of record lives here.** If the DB vanished, every stream and balance would be intact on-chain; users would just lose their saved names and drafts.

5.1 Stack

- **Neon serverless Postgres**, accessed via **Drizzle ORM** (`neon-http` driver).
- Every DB module is guarded by `import "server-only"`, so a client bundle that tries to import it fails the build. The database URL and all secrets stay server-side.
- API routes run on the Node runtime (`runtime = "nodejs"`, `dynamic = "force-dynamic"`) because auth depends on per-request headers.

5.2 Schema (`src/db/schema.ts`)

- `users` — `privyId` (Privy DID, unique; the login identity), `username` (lowercase, unique, with a `usernameChangedAt` for cooldown), `walletAddress`, `email`, `notificationEmail` (unique; a wallet-only

user's contact address — **never a login**), `displayName` , `role` , `settings` (jsonb, holds notification preferences).

- `payees` — a per-user address book (`ownerId` FK, label, address, role, note).
- `streamDrafts` — half-composed streams (`draft` / `committed` status, optional `onchainStreamId` once committed).

5.3 Query layer (`src/db/queries.ts`)

All DB access is funnelled through named, `server-only` functions so the invariants live in one place:

- `upsertUser` is the single entry point that resolves a Privy DID to an internal user row, creating it on first sign-in and syncing email/wallet on subsequent ones. It throws `EmailBoundElsewhereError` if the email is already owned by a different account — which prevents two Privy identities from silently colliding on one email (the client handles this by logging out and explaining; see §5.5).
- **Identity resolution** (the read path that powers `@handles` on stream cards):
 - `getUserByUsername` — forward-resolve a handle to a wallet (used by `/api/resolve` for "pay @alice").
 - `getPublicIdentitiesByAddresses` — reverse-resolve a batch of addresses to `{username, displayName}` (used to label the counterparty on each stream card).
 - `getUsersByUsernames` — batch forward-resolve for batch payments.
 - These only ever return *public* fields (handle, display name, wallet). Email is never exposed to the client.
- `getNotifiableByAddress` — address → email, used **server-side only** to decide who to email; the result never crosses to the browser.
- `setUsername` enforces a 14-day cooldown on changes (first set is free), and uniqueness is enforced both in code and by a Postgres unique constraint (`isUniqueViolation` catches error `23505`).

5.4 Auth (`src/app/api/_auth.ts` , `src/lib/privy-server.ts`)

- The client attaches a fresh **Privy access token** as a bearer on every `/api/*` call (`useApi` , §6.6).
- On the server, `verifyCaller` uses the Privy `PrivyClient` (`appId` + `appSecret` , server-only) to verify the token via `privy.verifyAuthToken` , then fetches the caller's linked email and wallet. The result is `Caller = { privyId, email, walletAddress }` .
- `requireUser(req)` wraps that plus `upsertUser` , returning either `{ user }` or `{ response }` (a 401/403 `NextResponse`). Every protected route starts with `const gate = await requireUser(req); if ("response" in gate) return gate.response;` .
- The app secret and database URL never leave the server; the client only ever holds a short-lived Privy token.

5.5 API routes (all auth-gated unless noted)

Route	Purpose	Reads
<code>/api/me</code> (GET/PATCH)	Upsert-on-read the profile; update display name/role/settings	Neon
<code>/api/me/email</code> (POST/DELETE)	Bind/clear a notification-only email	Neon

Route	Purpose	Reads
<code>/api/username</code> (GET/POST)	Check availability / set handle (cooldown)	Neon
<code>/api/resolve</code> (GET)	@handle → wallet address	Neon
<code>/api/resolve-addresses</code> (POST)	addresses → {handle, name} for card labels	Neon
<code>/api/resolve-batch</code> (POST)	mixed handles/addresses → wallets for batch pay	Neon
<code>/api/payees</code> (GET/POST), <code>/api/payees/[id]</code> (DELETE)	address book CRUD	Neon
<code>/api/drafts</code> (GET/POST), <code>/api/drafts/[id]</code> (DELETE)	stream draft CRUD	Neon
<code>/api/notify</code> (POST)	account-activity email dispatcher	Neon (recipient lookup)
<code>/api/faucet</code> (POST)	drip testnet USDC via Circle	Circle API
<code>/api/rpc</code> (POST)	same-origin JSON-RPC proxy (see §6.5)	Arc RPC

`/api/resolve` is a representative read: gate the caller → validate the handle → `getUserByUsername` → return only { `walletAddress`, `username`, `displayName` }, 404 if unknown or wallet-less. It never leaks anything private.

5.6 Faucet (`/api/faucet`)

Auth-gated. Given { `address` }, it calls Circle's drips endpoint (`CIRCLE_DRIPS_URL` , blockchain `ARC-TESTNET`) and returns one of { `dripped` }, { `rateLimited` }, or { `fallback` } (client should open the public Circle faucet). This is what powers the one-click "get testnet USDC" button so a demo attendee can fund a wallet without leaving the app.

5.7 Notifications (`/api/notify` , `src/lib/notifications.ts`)

An auth-gated event dispatcher. The client fires fire-and-forget events (`welcome` , `signin` , `stream_started` , `stream_activated` , `stream_claimed` , `stream_topped_up` , `stream_cancelled` , `request_received` , `counter_offer` , `receipt`); the server resolves the counterparty's email *server-side* (via `getNotifiableByAddress`), applies the recipient's opt-out preferences (`NOTIFICATION_CATEGORIES` , opt-out model — enabled by default), and sends via **Resend**. Sends are best-effort and never block or surface an error, because a notification is a courtesy layered on top of an action that already succeeded on-chain. Emails silently no-op if `RESEND_API_KEY` / `EMAIL_FROM` are unset. A wallet-only user's `notificationEmail` receives alerts but can never be used to log in.

6. Front-end read path (chain + database)

This section is limited to the components and hooks that *read* data, per scope. The presentational/animation-only code is omitted.

6.1 The on-chain read hooks (`src/hooks/usePayroll.ts`)

All reads use wagmi's `useReadContract` / `useReadContracts` with React Query underneath. Common query settings: `refetchInterval` of 3-5s, `placeholderData: keepPreviousData` (so a refetch never blanks the UI), and `enabled` gated on having an address.

- `useEmployerStreams(addr)` / `useEmployeeStreams(addr)` → call `getEmployerStreams` / `getEmployeeStreams`, returning that wallet's array of stream ids (5s poll). These drive the "my streams" lists on the payer/payee dashboards.
- `useStreamsMeta(ids)` → the workhorse. It maps the id list to one **batched multicall** of `streams(id)` reads (`useReadContracts`), so a dashboard with N streams does *one* round-trip, not N. Three resilience measures:

1. The contracts array is memoized on a **stable string key** (`idKey`) so the query key doesn't churn every render (the raw `ids` array identity changes each poll).
2. `keepPreviousData` across refetches.
3. A **per-id last-good-decode cache** (`useRef(Map)`) so if one sub-call of a poll fails transiently, that card falls back to its last good value instead of vanishing. (This fixed real card flicker.)

It decodes each `streams(id)` tuple into a typed `StreamMeta`.

- `useUsdcBalance(addr)` → ERC-20 `balanceOf` (3s poll) — the wallet balance strip.
- `useUsdcAllowance(owner)` → ERC-20 `allowance(owner, PAYROLL_ADDRESS)` (5s poll) — used to decide whether an approval step is needed before a create/top-up.
- `usePayerRequests` / `usePayeeRequests` / `useRequestsMeta` → the exact mirror of the stream hooks for the request/negotiation lifecycle (`getPayerRequests`, `getPayeeRequests`, batched `requests(id)` reads, same stable-key + last-good-cache pattern). `ReqStatus` is mirrored client-side as a const object matching the Solidity enum order.

6.2 Deriving money from the tuple (`src/lib/stream-math.ts`)

`streamMath(stream, nowSec)` is a **pure function** that turns the on-chain tuple into every figure the UI needs, with **no extra RPC calls**:

- `withdrawn`, `streamedSoFar` (= withdrawn + currently-accrued), `unclaimed` (accrued not yet withdrawn), `committed` (= totalDeposited), `remaining` (escrow left), `runwaySeconds`.
- Boolean phase flags: `notStarted`, `flowing`, `streaming`, `cancelled`.
- A `phase` enum: `scheduled` | `live` | `awaiting_claim` | `claimed` | `ended`.

It reimplements the contract's accrual (`elapsed * rate`, capped at deposit) so the client and chain agree. `cancelled` is detected by the exact on-chain signature described in §4.8: `!active && deposit === 0 && withdrawn < totalDeposited` (distinguishing a cancel from a clean full-claim, where `withdrawn == totalDeposited`). A `START_SKEW_SEC` (30s) tolerance smooths the scheduled→live flip against minor clock differences.

Because every displayed number is a deterministic function of data already fetched, the cards don't each fire their own reads — one batched poll feeds the whole page and `streamMath` does the rest locally.

6.3 The live ticker (`src/components/StreamTicker.tsx`)

This is the "money moving by the second" visual, and it is **purely cosmetic interpolation** — a point worth making explicitly in a demo. Given the last on-chain `accrued` value and the known `ratePerSecond`, it advances the displayed number every animation frame using `performance.now()` deltas. It:

- pauses when the tab is hidden (no wasted rAF churn or RPC pressure),
- honours `prefers-reduced-motion` (snaps to the real value, no animation),
- re-seeds from the real on-chain value on every 5s poll.

So the number you see ticking is a smooth *display* between two real reads; the truth is always the last struct fetched. It never hammers the RPC per frame. `useLiveCounter` is a simpler variant used only on the marketing hero (no chain data at all).

6.4 What the dashboards render (`src/app/payer/page.tsx` , `src/app/payee/page.tsx`)

Both pages: resolve the active address, list ids (`useEmployerStreams` / `useEmployeeStreams`), batch-decode via `useStreamsMeta`, and render `StreamCard` s that each call `streamMath` for their figures. The payer page adds two touches worth knowing:

- **Optimistic cancel:** when the payer cancels, the id is held in a `cancellingIds` set and that card is rendered as already-settled (`active:false`, `deposit:0`) *immediately* on tx submit, so the ticker stops the instant the wallet confirms rather than at the next 5s poll. The override is dropped after the post-confirm refetch, which already carries the real cancelled state. This works precisely because zeroing `deposit` while `withdrawn < totalDeposited` is the on-chain cancel signature, so the optimistic card matches what the refetch confirms.
- **Post-action flow:** write → `waitForTransactionReceipt` → `refetch()` → fire the notification with figures *snapshotted before* the write (e.g. the claimable amount before withdraw zeroes it), so the email reports what was actually moved; a rejected/failed tx fires no email.

`StreamCard` also drives the client-side `stream_activated` notification on the scheduled→live edge (a `localStorage` guard dedupes repeat mounts), since there's no on-chain event indexer — the notify is fired by whichever card observes the flip.

6.5 The RPC proxy — why browser reads go through `/api/rpc`

This is a subtle, important piece of production hardening (and a documented lesson from this build):

- The browser must **not** hit a raw `rpc.*.arc.io` host directly. The primary upstream fails browser CORS preflight, and the CORS-friendly mirrors get blocked by ad-blockers (`ERR_BLOCKED_BY_CLIENT`), which silently breaks reads for a chunk of users.
- So `browserSafeRpcHttp()` returns a **same-origin** `/api/rpc` URL in the browser, and the direct upstreams only on the server.
- `/api/rpc/route.ts` forwards the JSON-RPC body verbatim to `ARC_RPC_UPSTREAMS` (four providers: Arc, Blockdaemon, dRPC, QuickNode) with an 8s per-upstream timeout and failover, `no-store`. Same-origin means no CORS and nothing for an ad-blocker to match.
- wagmi is configured with a `fallback` transport over `browserSafeRpcHttp()` with `batch: true` and Multicall3 batching (`0xcA11...CA11`), 2s polling, `ssr: true`. So multiple `useReadContract` calls in a render collapse into a single multicall request through the proxy.

6.6 The database read path from the client (`src/hooks/useApi.ts` , `useProfile.ts`)

- `useApi` is a thin typed client for our own `/api/*` routes. Every call fetches a fresh Privy token and sends it as a bearer; nothing here ever sees the app secret or DB URL. It surfaces a typed `ApiError` carrying HTTP status and a machine-readable `code` so callers can branch (e.g. on `email_bound_elsewhere`).
 - `useProfile` mounts behind auth and calls `GET /api/me`, which upserts-on-read — so simply rendering it guarantees the account row exists. If the login's email is bound to another account, it catches `email_bound_elsewhere`, logs the user out, and surfaces an explanation rather than leaving a half-account.
 - Identity labels on cards come from `resolveAddresses` (reverse-resolve counterparties to `@handles`); paying by handle uses `resolveUsername` / `resolveBatch` (forward-resolve).
-

7. The write path (how mutations reach the chain)

Reads are the focus per scope, but two write-path decisions are load-bearing for reliability and worth being able to defend.

7.1 Static gas limits (no pre-flight estimation)

Every write passes an explicit `gas` limit from a static `GAS_LIMITS` table (e.g. `withdraw` 200k, `createStream` 600k, `acceptRequest` 650k; `batch = 150_000 + 300_000 * count`). Rationale:

- An injected wallet handed a tx with **no** gas field runs its own `eth_estimateGas` before it can render the confirm prompt — that round-trip is the popup lag users feel.
- Supplying `gas` up front makes the wallet skip estimation and prompt instantly.
- A previous version estimated on our own RPC first, which just moved the same round-trip onto our side (serial, up to ~1s) and, on timeout, fell back to no gas so the wallet estimated anyway — the worst of both. Static limits remove pre-flight entirely.
- **Safety:** gas is refunded — the sender pays `gasUsed × price`, never the limit — so overshooting costs nothing. Each limit is ~2.5-3x the measured max from `forge test --gas-report`, and the wallet still shows it as an editable max.

7.2 Pinned EIP-1559 fees (`ARC_FEE`)

Arc mandates `maxFeePerGas >= 20 Gwei` (its testnet minimum base fee) or a tx "may remain pending indefinitely or fail outright." The Privy embedded-wallet signer forwards fee fields verbatim without Arc-aware estimation, so a write with no fees set slips below the floor and the RPC rejects it. Every write therefore pins `maxFeePerGas = 50 Gwei` (2.5x the floor for headroom) and `maxPriorityFeePerGas = 2 Gwei`. Because `maxFeePerGas` is only a cap, the effective cost is still base+tip (about a cent). Injected wallets treat these as editable defaults, so it's a no-op for them.

7.3 Approval + batch flow

For a single create/top-up the client checks allowance (`useUsdcAllowance`) and inserts an `approve` if needed. For a batch, `useBatchCreateStreams` does exactly two signatures regardless of recipient count: one `approve` for the summed total, then one `createStreams`. Because the batch tx is atomic, there's no partial-success state to reconcile; progress is still surfaced as a per-recipient array (all flip together) so the existing batch UI is unchanged.

8. Statements and receipts (read-derived documents)

Cadence generates shareable, branded PDFs entirely from on-chain data:

- **Per-stream receipts** and **full account statements** are produced client-side with jsPDF + jsPDF-autotable ([src/lib/statement.ts](#)). Every figure is derived from the on-chain structs via `streamMath` — the document is a formatting of chain truth, not a separate ledger. It includes an anti-tamper backdrop and downloads as `cadence-statement-<addr>-<date>.pdf` .

So both sides always have a clean record of what was paid, reconstructable purely from the chain.

9. Demo defence — likely questions and crisp answers

"Where does the money actually live?" In the `PayrollManager` contract's escrow, per stream. The DB holds no balances. Wipe the DB and every balance is intact on-chain.

"Is the ticking number real money?" The *last read* is real on-chain accrued value; the ticking between reads is cosmetic interpolation at the known rate, re-seeded every 5s. The withdrawable amount is always computed by the contract at withdraw time.

"What stops the contract from owing more than it holds?" `_accrued` caps at the remaining deposit, and a full-drain withdrawal takes everything — proven by an invariant test. No dust strands.

"What happens on cancel?" Accrued-to-date goes to the employee, the rest refunds to the employer, atomically, in one tx. State becomes `active:false, deposit:0, withdrawn<totalDeposited` , which the UI reads as "cancelled."

"Why can you batch-create but a multicall can't?" `createStream` binds employer and funds to `msg.sender` ; through an aggregator that'd be the aggregator. Batching lives in the contract so the payer stays the caller — one approval, one atomic tx, all-or-nothing.

"What does top-up do to a live stream?" Holds the finish date fixed and raises the rate, preserving already-owed amounts by re-anchoring the accrual clock. On a scheduled stream it's additive; on a drained one it reactivates from now.

"How does the request negotiation avoid trust?" It's an on-chain state machine. A counter-offer escrows the payer's funds immediately, so the payee accepts with one call and no further payer signature; a 6-hour window plus a permissionless reclaim bounds the escrow.

"Why Arc and not an L2?" Single-asset USDC (earnings are spendable without a separate gas token), sub-second deterministic finality (cash-out-now is real, not a 7-day challenge window), and stable sub-cent fees (micro-withdrawals stay economical). All three are required; no other EVM chain gives you all three.

"Is this production/mainnet?" It runs on **Arc testnet** (chain 5042002) with testnet USDC via the Circle faucet. The contract has no admin and no upgrade path; a mainnet deployment would be the same code plus an audit.

"Any admin backdoor?" None. No owner, no pause, no upgrade, no privileged withdrawal. The deployer has no special powers.

10. File map (where to look)

- `contracts/src/PayrollManager.sol` — the entire contract.
- `contracts/src/utils/ReentrancyGuard.sol` — minimal guard.
- `contracts/test/PayrollManager.t.sol` — 40 tests.
- `contracts/script/Deploy.s.sol` — deploy script.
- `src/hooks/usePayroll.ts` — all on-chain reads and writes, gas/fee strategy.
- `src/lib/stream-math.ts` — pure chain→UI derivation.
- `src/components/StreamTicker.tsx` , `StreamCard.tsx` — read-path display.
- `src/app/payer/page.tsx` , `src/app/payee/page.tsx` — dashboards.
- `src/lib/chains.ts` , `wagmi.ts` , `rpc-endpoints.ts` , `src/app/api/rpc/route.ts` — chain config + RPC proxy.
- `src/db/schema.ts` , `queries.ts` , `index.ts` — off-chain identity layer.
- `src/app/api/_auth.ts` , `src/lib/privy-server.ts` — auth.
- `src/app/api/*` — identity resolution, faucet, notify, drafts, payees.
- `src/lib/statement.ts` — receipts/statements from chain data.

Document scope: contract and backend implementation, plus front-end components that read from the chain or database. Presentational and animation-only front-end code is intentionally out of scope.